```
================================================================================
🧪 ЗАПУСК ГЛОБАЛЬНОГО ТЕСТА: CAUSAL CONV1D (FORWARD + BACKWARD)
================================================================================
Тестирование на устройстве: NVIDIA GeForce RTX 2060

Вычисление CausalConv1d Forward на GPU...
Вычисление CausalConv1d Backward на GPU...

Верификация расчетов с оригинальным PyTorch...
  ✅ Forward Conv1d: Полное совпадение карт признаков свертки!
  ✅ Backward (Grad X): Градиенты по входному сигналу полностью совпали!
C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_nncl_glob.py:238: UserWarning:
The .grad attribute of a Tensor that is not a leaf Tensor is being accessed. Its .grad attribute won't be
populated during autograd.backward(). If you indeed want the .grad field to be populated for a non-
leaf Tensor, use .retain_grad() on the non-leaf Tensor. If you access the non-leaf Tensor by mistake,
make sure you access the leaf Tensor instead. See github.com/pytorch/pytorch/pull/30531 for more
information. (Triggered internally at
C:\actions-runner\_work\pytorch\pytorch\build\aten\src\ATen/core/TensorBody.h:498.)
  np.testing.assert_allclose(cl_res_grad_W, t_W.grad.squeeze(1).numpy(), rtol=1e-4, atol=1e-4)
Traceback (most recent call last):
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_nncl_glob.py", line 252,
in <module>
    test_causal_conv1d_glob_pipeline() # <-- Добавили новый шаг
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_nncl_glob.py", line 238,
in test_causal_conv1d_glob_pipeline
    np.testing.assert_allclose(cl_res_grad_W, t_W.grad.squeeze(1).numpy(), rtol=1e-4, atol=1e-4)
                                              ^^^^^^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'squeeze'
```

Теперь сбой видимо самого pytorch?

```python
import torch.nn.functional as F

def test_causal_conv1d_glob_pipeline():
    print("\n" + "="*70)
    print("🧪 ЗАПУСК ГЛОБАЛЬНОГО ТЕСТА: CAUSAL CONV1D (FORWARD + BACKWARD)")
    print("="*70)
    print(f"Тестирование на устройстве: {devices[0].name}")

    Batch, Channels, Length = 2, 8, 16
    kernel_size = 4

    np.random.seed(42)
    X_init = np.random.randn(Batch, Channels, Length).astype(np.float32)
    W_init = np.random.randn(Channels, kernel_size).astype(np.float32)
    B_init = np.random.randn(Channels).astype(np.float32)

    # Создаем тензоры в OpenCL
    x_cl = Tensor.from_numpy(X_init, ctx, queue)
    w_cl = Tensor.from_numpy(W_init, ctx, queue)
    b_cl = Tensor.from_numpy(B_init, ctx, queue)

    print("\nВычисление CausalConv1d Forward на GPU...")
    y_cl = nnclglob.causal_conv1d(x_cl, w_cl, b_cl)

    # Входящий градиент ошибок
    grad_output_init = np.random.randn(Batch, Channels, Length).astype(np.float32)
    grad_output_cl = Tensor.from_numpy(grad_output_init, ctx, queue)

    print("Вычисление CausalConv1d Backward на GPU...")
    grad_X_cl, grad_W_cl, grad_B_cl = nnclglob.causal_conv1d_backward(grad_output_cl, x_cl, w_cl)

    # Выкачиваем результаты
    cl_res_y = HostReturner.to_cpu(y_cl)
    cl_res_grad_X = HostReturner.to_cpu(grad_X_cl)
    cl_res_grad_W = HostReturner.to_cpu(grad_W_cl)
    cl_res_grad_B = HostReturner.to_cpu(grad_B_cl)

    # ЭТАЛОННЫЙ РАСЧЕТ НА PYTORCH
    print("\nВерификация расчетов с оригинальным PyTorch...")
    t_X = torch.tensor(X_init, requires_grad=True)
    t_W = torch.tensor(W_init, requires_grad=True).unsqueeze(1) # PyTorch ждет [Channels, 1, K] для depthwise
    t_B = torch.tensor(B_init, requires_grad=True)

    # Прямой ход PyTorch с каузальным паддингом
    x_padded = F.pad(t_X, (kernel_size - 1, 0))
    t_Y = F.conv1d(x_padded, t_W, bias=t_B, groups=Channels)

    # Обратный ход PyTorch
    t_grad_output = torch.tensor(grad_output_init)
    t_Y.backward(t_grad_output)
```

```python
    t_B = torch.tensor(B_init, requires_grad=True)

    # Прямой ход PyTorch с каузальным паддингом
    x_padded = F.pad(t_X, (kernel_size - 1, 0))
    t_Y = F.conv1d(x_padded, t_W, bias=t_B, groups=Channels)

    # Обратный ход PyTorch
    t_grad_output = torch.tensor(grad_output_init)
    t_Y.backward(t_grad_output)

    # СРАВНЕНИЕ РЕЗУЛЬТАТОВ
    np.testing.assert_allclose(cl_res_y, t_Y.detach().numpy(), rtol=1e-4, atol=1e-4)
    print("  ✅ Forward Conv1d: Полное совпадение карт признаков свертки!")

    np.testing.assert_allclose(cl_res_grad_X, t_X.grad.numpy(), rtol=1e-4, atol=1e-4)
    print("  ✅ Backward (Grad X): Градиенты по входному сигналу полностью совпали!")

    np.testing.assert_allclose(cl_res_grad_W, t_W.grad.squeeze(1).numpy(), rtol=1e-4, atol=1e-4)
    print("  ✅ Backward (Grad Weight): Градиенты по весам фильтров полностью совпали!")

    np.testing.assert_allclose(cl_res_grad_B, t_B.grad.numpy(), rtol=1e-4, atol=1e-4)
    print("  ✅ Backward (Grad Bias): Градиенты по смещениям полностью совпали!")

    print("\n🎉 ТЕСТ КАУЗАЛЬНОЙ СВЕРТКИ УСПЕШНО ПРОЙДЕН!")
    print("="*70 + "\n")


if __name__ == "__main__":
    test_rmsnorm_glob_pipeline()
    test_softmax_glob_pipeline()
    test_bmm_glob_pipeline()
    test_causal_conv1d_glob_pipeline() # <-- Добавили новый шаг
```

Следующие действия — хитрый слой

Мы реализовали много слоев ,но остался 1 очень важный но хзит рый слой — dropout .

Он в отличии от реализованных будет выполняться на CPU вне зависимости от того где запущенна программа. Чтобы не гонять всю память мы будем использовать opencl.writebuffer (offset=<позиция элемента>,data=<мыссив из 1 нуля>, length=sizeof(float)) . а процессор будет определять вероятность зануления и индекс занулемого элемента.